

Aprendizaje de Máquina

Sesión 9: Aprendizaje no supervisado

Angel Rodríguez

Ajuste de Syllabus

Tema 1. Fundamentos del aprendizaje de máquina

- 1.1 Concepto de aprendizaje de máquina
- 1.2 Aprendizaje supervisado
- 1.3 Aprendizaje no supervisado
- 1.4 Problemas de clasificación, regresión y agrupamiento

Tema 2. Teoría del aprendizaje

- 2.1 Flujos generales de trabajo
- 2.2 Aprendizaje y error
- 2.3 Riesgo empírico y riesgo estructural
- 2.4 Complejidad y dimensionalidad
- 2.5 Ruido, sobreajuste y generalización

Tema 3. Discriminantes y modelos lineales

- 3.1 El perceptrón
- 3.2 Separabilidad lineal
- 3.3 Regresión lineal
- 3.4 Entrenamiento y evaluación de modelos lineales

Tema 4. ~~Perceptrón multicapa y redes neuronales~~

- 4.1 Introducción a las redes neuronales
- 4.2 Perceptrón multicapa
- 4.3 Funciones de activación
- 4.4 Algoritmo de retropropagación
- 4.5 Aplicaciones del perceptrón multicapa

Tema 5. Aprendizaje no supervisado

- 5.1 Agrupamiento basado en distancia
- 5.2 Agrupamiento jerárquico
- 5.3 Evaluación de agrupamientos
- 5.4 Reducción de dimensiones

Tema 6. Aprendizaje de máquina para textos

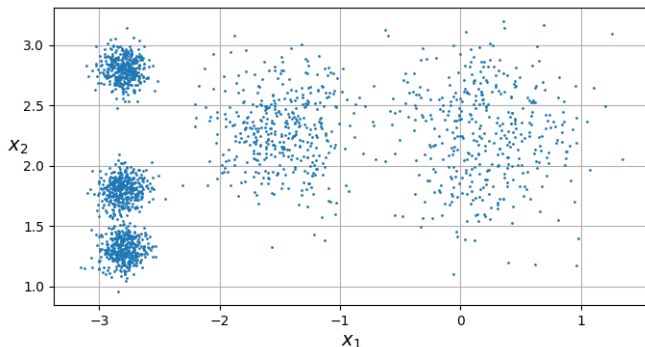
- 6.1 Introducción al procesamiento de textos
- 6.2 Representación numérica de textos
- 6.3 Agrupamiento de textos
- 6.4 Modelos de clasificación de texto
- 6.5 Evaluación y selección de modelos

Syllabus personalizado a sus intereses

- Basado en la clase pasada, después de aprendizaje no supervisado y procesamiento de textos, y si el tiempo lo permite, veremos:
 1. **Descenso de gradiente estocástico.**
Entrenamiento eficiente con grandes conjuntos de datos; p. ej., modelos con millones de observaciones.
 2. **Árboles de decisión y ensambles de árboles.**
Clasificación y regresión con datos tabulares; p. ej., predecir si un cliente dejará un servicio.
 3. **Métodos de kernel.**
Clasificación no lineal; p. ej., separar clases que no son linealmente separables.
 4. ~~**Métodos lineales para regresión/clasificación.**~~
~~Predicción de variables continuas; p. ej., predecir el precio de una vivienda/Predicción de categorías; p. ej., clasificar un correo como spam o no spam. A profundidad, teórico o aplicado rápido.~~

K-means (1957)

- Supongamos que queremos agrupar en K clusters.
- K -means intentará obtener una distribución óptima de cómo ubicar K centros en los datos.
- Queremos que la distancia intra-clusters sea pequeña y la distancia inter-clusters sea grande.



K-means: idea general

- Queremos dividir un conjunto de datos

$$\{\mathbf{x}_1, \dots, \mathbf{x}_N\}$$

en K grupos o **clusters**.

- Cada cluster está representado por un centroide

$\mu_1, \dots, \mu_K.$

- La idea es asignar cada punto al centroide más cercano.

Función objetivo

K-means minimiza la suma de distancias cuadradas dentro de cada cluster (*medida de distorsión*):

$$J = \sum_{n=1}^N \sum_{k=1}^K r_{nk} \|\mathbf{x}_n - \boldsymbol{\mu}_k\|^2.$$

donde

$$r_{nk} = \begin{cases} 1, & \text{si } \mathbf{x}_n \text{ pertenece al cluster } k, \\ 0, & \text{en otro caso.} \end{cases}$$

Meta: Encontrar $\{r_{nk}\}$ y $\{\mu_k\}$ que minimicen J .

¿Cómo minimizar J ?

Haciendo un proceso iterativo de dos pasos:

1. Teniendo los μ_k s, minimizamos J con respecto a los r_{nk} s.
2. Teniendo los r_{nk} s, minimizamos J con respecto a los μ_k s.

Hasta que el algoritmo converja, es decir hasta que no cambien los valores.

Paso 1: asignación de clusters (encontrar $\{r_{nk}\}$)

Si los centroides $\mu_1, \dots, \mu_K$ están fijos, para optimizar J con respecto a los r_{nk} s, asignamos cada punto al centroide más cercano:

$$r_{nk} = \begin{cases} 1, & \text{si } k = \arg \min_j \|\mathbf{x}_n - \boldsymbol{\mu}_j\|^2, \\ 0, & \text{en otro caso.} \end{cases}$$

Es decir, cada observación se asigna al cluster cuyo centroide está más cerca.

Paso 2: actualización de centroides (encontrar $\{\mu_k\}$)

Si las asignaciones r_{nk} están fijas, para optimizar J con respecto a los μ_k s, actualizamos cada centroide como el promedio de los puntos asignados a ese cluster:

$$\mu_k = \frac{\sum_{n=1}^N r_{nk} \mathbf{x}_n}{\sum_{n=1}^N r_{nk}}.$$

Por eso el centroide representa el “centro” de los puntos del cluster.

Algoritmo K-means

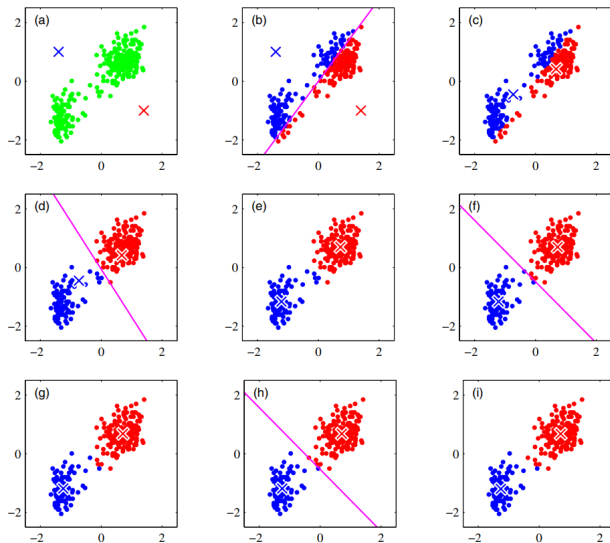
1. Elegir el número de clusters K .
2. Inicializar los centroides $\mu_1, \dots, \mu_K$.
3. Asignar cada punto al centroide más cercano.
4. Recalcular cada centroide como el promedio de sus puntos.
5. Repetir los pasos 3 y 4 hasta que las asignaciones ya no cambien.

Convergencia

- En cada paso, la función objetivo J no aumenta.
- El paso de asignación reduce J al escoger el centroide más cercano.
- El paso de actualización reduce J al mover el centroide al promedio.
- Por lo tanto, el algoritmo converge.

Importante: K-means puede converger a un mínimo local, por lo que la inicialización importa.

Interpretación geométrica



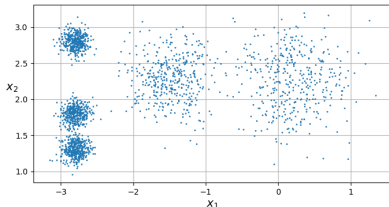
K-means en Scikit-Learn

Supongamos que tengo estos datos:

```
import numpy as np
from sklearn.datasets import make_blobs

blob_centers = np.array([[ 0.2, 2.3], [-1.5 , 2.3], [-2.8, 1.8], [-2.8,
                        2.8], [-2.8, 1.3]])
blob_std = np.array([0.4, 0.3, 0.1, 0.1, 0.1])
X, y = make_blobs(n_samples=2000, centers=blob_centers, cluster_std=
                  blob_std, random_state=7)

plt.scatter(X[:, 0], X[:, 1], s=1)
```



K-means en Scikit-Learn

Usando *K*-means:

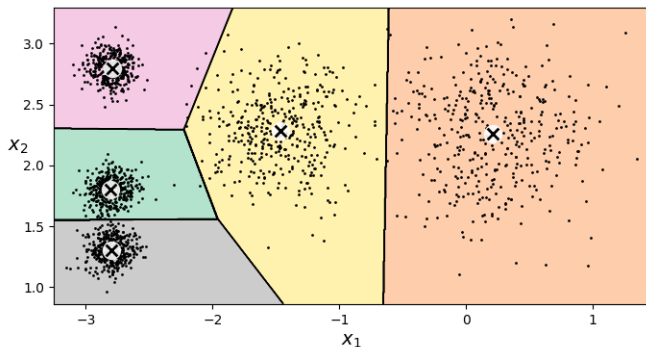
```
from sklearn.cluster import KMeans

k = 5
kmeans = KMeans(n_clusters=k, random_state=43)
y_pred = kmeans.fit_predict(X)

# or just kmeans.fit(X)

kmeans.cluster_centers_
```

K-means en Scikit-Learn



Si tengo nuevos puntos:

```
X_new = np.array([[0, 2], [3, 2], [-3, 3], [-3, 2.5]])
kmeans.predict(X_new)
```

Inercia

Podemos correr K -means con diferentes semillas y quedarnos con el mejor cluster...

¿Cuál es el mejor cluster?

¡Necesitamos definir una métrica! La *inercia* está definida como:

$$\text{inercia} = \sum_k^K \sum_n^N r_{nk} \|x_n - \mu_j\|^2,$$

y es la suma de todas las distancias al cuadrado entre cada punto y su centroide más cercano.

Fórmula más intuitiva:

$$\text{inercia} = \sum_k^K \sum_{\mathbf{x} \in C_k} \|\mathbf{x} - \mu_k\|^2,$$

Inercia en Scikit-Learn

kmeans.inertia_

Inicialización con *k-means++*

La inicialización *k-means++* busca elegir centroides iniciales bien separados entre sí.

1. Elegir el primer centroide μ_1 de forma aleatoria del conjunto de datos.
2. Para elegir un nuevo centroide μ_i , seleccionar un punto $\mathbf{x}_i$ con probabilidad

$$\frac{D(\mathbf{x}_i)^2}{\sum_{n=1}^N D(\mathbf{x}_n)^2},$$

donde $D(\mathbf{x}_i)$ es la distancia entre $\mathbf{x}_i$ y el centroide ya seleccionado más cercano.

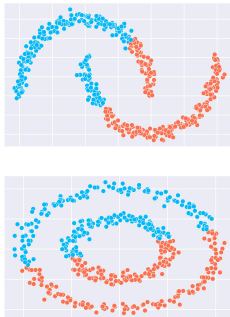
3. Repetir el paso anterior hasta haber seleccionado los K centroides iniciales $\mu_1, \dots, \mu_K$.

Idea: los puntos más alejados de los centroides ya elegidos tienen mayor probabilidad de convertirse en nuevos centroides.

Ventajas y desventajas de *K*-means

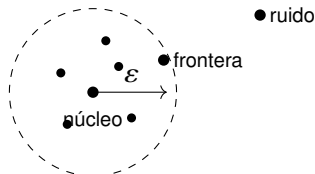
- + Es rápido
- + Escala fácil
- Seleccionar K no es una tarea fácil.
- Tiene simetría esférica. Clusters de diferentes formas pueden dar malos resultados:

KMeans



DBSCAN: tipos de puntos

- **Punto núcleo (*core point*):** Tiene al menos `min_samples` observaciones dentro de su vecindad- ϵ .
- **Punto frontera (*border point*):** No tiene suficientes vecinos para ser núcleo, pero se encuentra dentro de la vecindad- ϵ de un punto núcleo.
- **Ruido (*noise*):** No es punto núcleo ni se encuentra en la vecindad de uno.



Agrupamiento para segmentación de imágenes

Existen diferentes tipos de segmentación de imágenes:

- **Segmentación de colores.** Píxeles con colores similares son asignados al mismo segmento. Por ejemplo, analizar imágenes satelitales para medir el total de bosque en una región.
- **Segmentación semántica.** Todos los píxeles que son parte del mismo objeto son asignados al mismo segmento. Por ejemplo, en carros autónomos, se pueden agrupar los peatones.
- **Segmentación de instancias.** Todos los píxeles que son parte de un objeto individual son asignados al mismo segmento. En este caso habría diferentes grupos para cada peatón.

K-means para segmentación de color.

Práctica 1: Segmentación de imágenes con K-means

Objetivo: utilizar K-means para realizar segmentación por color de una imagen elegida por ti.

Cada píxel de una imagen RGB puede representarse como un vector

$$\mathbf{x}_i = \begin{bmatrix} R_i & G_i & B_i \end{bmatrix}.$$

La idea es agrupar píxeles con colores similares utilizando K-means.

Pregunta:

¿Qué ocurre con una imagen cuando reducimos el número de colores disponibles?

Práctica 1: Selecciona una imagen

1. Busca una imagen cualquiera en internet.
2. Descarga la imagen a tu computadora.
3. Preferentemente utiliza una imagen con:
 - varios colores,
 - distintos objetos o regiones,
 - buena resolución.
4. Sube la imagen a Google Colab.

Puede ser, por ejemplo:

- un paisaje,
- un animal (catarina en una hoja),
- una ciudad,
- comida,
- una fotografía satelital.

Práctica 1: Representar los píxeles

Una imagen RGB tiene dimensiones

(alto, ancho, 3).

Cada píxel contiene tres valores:

 $(R, G, B).$

Transforma la imagen en una matriz donde cada fila sea un píxel:

```
X = image.reshape(-1, 3)
```

```
print(X.shape)
```

Por lo tanto,

$$\mathbf{X} \in \mathbb{R}^{N \times 3},$$

donde N es el número total de píxeles.

Práctica 1: Construir la imagen segmentada

Reemplaza cada píxel por el centroide correspondiente a su cluster:

```
segmented_img = \
    kmeans.cluster_centers_[kmeans.labels_]

segmented_img = segmented_img.reshape(image.shape)

segmented_img = segmented_img.astype(np.uint8)
```

Visualiza el resultado:

```
plt.imshow(segmented_img)
plt.axis("off")
plt.show()
```

Práctica 1: Experimento

Repita el procedimiento utilizando

$$K \in \{2, 3, 4, 5, 6, 7, 8, 10\}.$$

Para cada valor de K :

1. Genera la imagen segmentada.
2. Compara con la imagen original.
3. Observa qué colores o regiones desaparecen.
4. Observa qué regiones comienzan a distinguirse al aumentar K .

Pregunta:

¿Qué efecto tiene el número de clusters sobre la imagen?

Práctica 1: Preguntas

Contesta brevemente:

1. ¿Qué representa un cluster en este problema?
2. ¿Qué representa cada centroide?
3. ¿Qué ocurre cuando K es muy pequeño?
4. ¿Qué ocurre al aumentar K ?
5. ¿Qué valor de K consideras más adecuado para tu imagen?
6. ¿K-means está identificando objetos o únicamente colores?

K-means para aprendizaje semi supervisado

Práctica 2: Experimento 1 — Baseline

Entrena un clasificador utilizando solamente las primeras 50 observaciones etiquetadas:

```
from sklearn.linear_model import \
    LogisticRegression

log_reg = LogisticRegression(
    max_iter=10_000
)

log_reg.fit(
    X_train[:n_labeled],
    y_train[:n_labeled]
)

accuracy = log_reg.score(
    X_test,
    y_test
)

print(accuracy)
```

Práctica 2: ¿Podemos elegir mejores ejemplos?

En el experimento anterior utilizamos simplemente las primeras 50 observaciones.

Sin embargo, si solamente podemos etiquetar un número pequeño de observaciones, nos conviene seleccionar ejemplos que sean **representativos** del conjunto de datos.

La idea será:

1. Agrupar los datos utilizando K-means.
2. Encontrar una observación representativa de cada cluster.
3. Utilizar esas observaciones para entrenar el clasificador.

Práctica 2: Experimento 2 — Clustering

Agrupar las 1,400 observaciones de entrenamiento en 50 clusters:

```
from sklearn.cluster import KMeans

k = 50

kmeans = KMeans(
    n_clusters=k,
    random_state=42
)

X_digits_dist = kmeans.fit_transform(
    X_train
)
```

El resultado `X_digits_dist` contiene las distancias de cada observación a cada uno de los 50 centroides.

Práctica 2: Ejemplos representativos

Para cada cluster, selecciona la observación más cercana a su centroide:

```
representative_digit_idx = \
    X_digits_dist.argmax(axis=0)

X_representative_digits = \
    X_train[representative_digit_idx]
```

Ahora tienes exactamente

50

imágenes representativas, una por cada cluster.

Visualiza estas 50 imágenes antes de continuar.

Práctica 2: Visualiza las 50 imágenes.

```
import matplotlib.pyplot as plt

plt.figure(figsize=(8, 2))
for index, X_representative_digit in enumerate(X_representative_digits):
    plt.subplot(k // 10, 10, index + 1)
    plt.imshow(X_representative_digit.reshape(8, 8), cmap="binary",
               interpolation="bilinear")
    plt.axis('off')

plt.show()
```


Práctica 2: Entrenar con ejemplos representativos

Entrena nuevamente la regresión logística:

```
log_reg = LogisticRegression(
    max_iter=10_000
)

log_reg.fit(
    X_representative_digits,
    y_representative_digits
)

accuracy_representative = \
    log_reg.score(
        X_test,
        y_test
    )

print(accuracy_representative)
```


Práctica 2: Analiza el resultado

Contesta:

- ¿Cuál fue la exactitud utilizando las primeras 50 observaciones?
- ¿Cuál fue la exactitud utilizando las 50 observaciones representativas?
- ¿Qué estrategia funcionó mejor?
- ¿Por qué seleccionar observaciones cercanas a los centroides puede producir un mejor conjunto de entrenamiento?

La cantidad de etiquetas utilizadas sigue siendo la misma:

50.

Lo único que cambió fue **qué observaciones etiquetadas utilizamos.**

Práctica 2: Experimento 3 — Propagación de etiquetas

Supón ahora que todos los ejemplos pertenecientes a un mismo cluster tienen la etiqueta de su ejemplo representativo.

Propaga las etiquetas:

```
y_train_propagated = np.empty(
    len(X_train),
    dtype=np.int64
)

for i in range(k):
    y_train_propagated[
        kmeans.labels_ == i
    ] = y_representative_digits[i]
```

Ahora tienes una etiqueta estimada para las 1,400 observaciones de entrenamiento.

Práctica 2: Entrenar con etiquetas propagadas

Entrena nuevamente el clasificador:

```
log_reg = LogisticRegression(
    max_iter=10_000
)

log_reg.fit(
    X_train,
    y_train_propagated
)

accuracy_propagated = \
    log_reg.score(
        X_test,
        y_test
    )

print(accuracy_propagated)
```

Compara este resultado con los dos experimentos anteriores.

Práctica 2: Experimento 4 — Propagación parcial

En lugar de propagar la etiqueta a **todo** el cluster, propágala solamente a las observaciones más cercanas al centroide.

Conserva únicamente el

50 %

de las observaciones más cercanas al centroide de cada cluster.

La idea es eliminar observaciones alejadas del centroide, que pueden ser menos representativas del cluster.

Práctica 2: Distancia al centroide

Obtén la distancia de cada observación al centroide de su propio cluster:

```
percentile_closest = 50

X_cluster_dist = X_digits_dist[
    np.arange(len(X_train)),
    kmeans.labels_
]
```

Después, para cada cluster, calcula el percentil 50 de las distancias y elimina las observaciones que se encuentren más lejos.

Práctica 2: Propagación parcial

```
for i in range(k):  
  
    in_cluster = (  
        kmeans.labels_ == i  
    )  
  
    cluster_dist = \  
        X_cluster_dist[in_cluster]  
  
    cutoff_distance = np.percentile(  
        cluster_dist,  
        percentile_closest  
    )  
  
    above_cutoff = (  
        X_cluster_dist > cutoff_distance  
    )  
  
    X_cluster_dist[  
        in_cluster & above_cutoff  
    ] = -1
```


Práctica 2: Compara los cuatro experimentos

Construye una tabla con tus resultados:

Método	Accuracy
50 observaciones iniciales	?
50 observaciones representativas	?
Propagación completa	?
Propagación parcial (50 %)	?

Identifica cuál de los cuatro métodos produjo la mayor exactitud.

Práctica 2: Preguntas

Contesta brevemente:

1. ¿Por qué las 50 observaciones representativas pueden ser mejores que las primeras 50 observaciones?
2. ¿Qué representa una imagen representativa de un cluster?
3. ¿Qué supuesto hacemos al propagar la etiqueta de la imagen representativa a todo el cluster?
4. ¿Por qué este supuesto puede fallar?
5. ¿Por qué conservar únicamente las observaciones cercanas al centroide puede mejorar los resultados?
6. ¿Qué método obtuvo la mayor exactitud?

Práctica 2: Entregable

Contesta las preguntas anteriores en la actividad de blackboard, es decir

1. Una tabla comparando las exactitudes obtenidas.
2. Las respuestas a las preguntas de análisis.

DBSCAN

DBSCAN significa:

Density-Based Spatial Clustering of Applications with Noise

La idea principal es identificar clusters con **regiones continuas de alta densidad, separadas por regiones de baja densidad.**

A diferencia de K-means:

- no necesitamos especificar el número de clusters,
- puede identificar clusters con formas no esféricas,
- puede detectar observaciones anómalas.

DBSCAN: vecindario de una observación

Para cada observación $\mathbf{x}_i$, DBSCAN considera todas las observaciones que se encuentran a una distancia menor o igual que

$$\varepsilon.$$

Este conjunto se llama el **vecindario** ε :

$$N_{\varepsilon}(\mathbf{x}_i) = \{\mathbf{x}_j : d(\mathbf{x}_i, \mathbf{x}_j) \leq \varepsilon\}.$$

El parámetro ε controla el tamaño del vecindario.

DBSCAN: observaciones núcleo

Una observación es una **observación núcleo** (*core instance*) si en su vecindario ε hay al menos

`min_samples`

observaciones, incluyendo a la propia observación.

Por lo tanto:

$$|N_{\varepsilon}(\mathbf{x}_i)| \geq \text{min_samples}$$



$\mathbf{x}_i$ es una observación núcleo.

Las observaciones núcleo se encuentran en regiones de **alta densidad**.

DBSCAN: formación de los clusters

DBSCAN construye los clusters a partir de las observaciones núcleo.

- Todas las observaciones dentro del vecindario de una observación núcleo pertenecen al mismo cluster.
- El vecindario puede contener otras observaciones núcleo.
- Una secuencia de observaciones núcleo vecinas puede formar un cluster grande y con una forma arbitraria.

Por esta razón, DBSCAN no necesita representar cada cluster mediante un centroide.

DBSCAN: anomalías

Una observación se considera una **anomalía** si:

- no es una observación núcleo, y
- no pertenece al vecindario de ninguna observación núcleo.

En Scikit-Learn, estas observaciones reciben la etiqueta

-1 .

Esto permite que DBSCAN pueda realizar simultáneamente:

- clustering,
- detección de observaciones aisladas.

Algoritmo DBSCAN

De manera simplificada:

1. Para cada observación, encuentra su vecindario ε .
2. Determina si es una observación núcleo utilizando `min_samples`.
3. Conecta observaciones núcleo cuyos vecindarios se intersectan.
4. Asigna al mismo cluster todas las observaciones conectadas.
5. Las observaciones que no pertenecen a ningún cluster se consideran anomalías.

DBSCAN en Scikit-Learn

Utilicemos el conjunto de datos `make_moons`:

```
from sklearn.cluster import DBSCAN
from sklearn.datasets import make_moons

X, y = make_moons(
    n_samples=1000,
    noise=0.05,
    random_state=42
)

dbscan = DBSCAN(
    eps=0.05,
    min_samples=5
)

dbscan.fit(X)
```

Etiquetas encontradas por DBSCAN

Después de entrenar:

dbscan.labels_

podemos obtener, por ejemplo:

```
array([0, 2, -1, -1, 1, 0, ..., 3])
```

Los valores

 $0, 1, 2, \dots$

representan los clusters encontrados.

La etiqueta

-1

representa una anomalía.

Observaciones núcleo

Scikit-Learn guarda los índices de las observaciones núcleo en:

dbscan.core_sample_indices_

Y las observaciones núcleo se encuentran en:

dbscan.components_

Por ejemplo:

components_

contiene las coordenadas de las observaciones que DBSCAN identificó como pertenecientes a regiones densas.

El efecto de ε

El valor de ε tiene un efecto muy importante.

Si

ε

es demasiado pequeño:

- aparecen muchos clusters pequeños,
- muchas observaciones pueden clasificarse como anomalías.

Si aumentamos ε :

- los vecindarios se hacen más grandes,
- más observaciones pueden conectarse,
- los clusters pueden fusionarse.

Función para graficar el resultado:

```
def plot_dbscan(dbscan, X, size, show_xlabels=True, show_ylabels=True):
    core_mask = np.zeros_like(dbscan.labels_, dtype=bool)
    core_mask[dbscan.core_sample_indices_] = True
    anomalies_mask = dbscan.labels_ == -1
    non_core_mask = ~(core_mask | anomalies_mask)

    cores = dbscan.components_
    anomalies = X[anomalies_mask]
    non_cores = X[non_core_mask]

    plt.scatter(cores[:, 0], cores[:, 1],
                c=dbscan.labels_[core_mask], marker='o', s=size, cmap="Paired")
    plt.scatter(cores[:, 0], cores[:, 1], marker='*', s=20,
                c=dbscan.labels_[core_mask])
    plt.scatter(anomalies[:, 0], anomalies[:, 1],
                c="r", marker="x", s=100)
    plt.scatter(non_cores[:, 0], non_cores[:, 1],
                c=dbscan.labels_[non_core_mask], marker=".")
    if show_xlabels:
        plt.xlabel("$x_1$")
    else:
        plt.tick_params(labelbottom=False)
    if show_ylabels:
        plt.ylabel("$x_2$", rotation=0)
    else:
        plt.tick_params(labelleft=False)
    plt.title(f"eps={dbscan.eps:.2f}, min_samples={dbscan.min_samples}")
    plt.grid()
    plt.gca().set_axisbelow(True)
```

Graficar el resultado

```
import matplotlib.pyplot as plt
import numpy as np

plot_dbscan(dbscan, X, size=100)
plt.show()
```


DBSCAN vs. K-means

	K-means	DBSCAN
Número de clusters	Se especifica	Se determina
Centroide	Sí	No
Formas arbitrarias	Limitado	Sí
Anomalías	No	Sí
Basado en	Distancia al centro	Densidad

DBSCAN puede funcionar especialmente bien cuando los clusters tienen formas complejas y están separados por regiones de baja densidad.

¿Cómo predecimos nuevas observaciones?

Una particularidad de la implementación de DBSCAN en Scikit-Learn es que no tiene un método

predict().

DBSCAN encuentra los clusters de los datos utilizados durante el entrenamiento, pero no define directamente cómo asignar una observación nueva.

Si necesitamos hacerlo, podemos entrenar posteriormente otro clasificador utilizando los clusters encontrados por DBSCAN.

Ejemplo: utilizar KNN después de DBSCAN

Podemos entrenar un clasificador KNN utilizando las observaciones núcleo:

```
from sklearn.neighbors import \
    KNeighborsClassifier

knn = KNeighborsClassifier(
    n_neighbors=50
)

knn.fit(
    dbscan2.components_,
    dbscan2.labels_[
        dbscan2.core_sample_indices_
    ]
)
```

Ahora KNN puede utilizar los clusters encontrados por DBSCAN para clasificar nuevas observaciones.

Graficar regiones de decisión

```
def plot_decision_boundaries(clusterer, X, resolution=1000, show_centroids=True,
                             show_xlabel=True, show_ylabel=True):
    mins = X.min(axis=0) - 0.1
    maxs = X.max(axis=0) + 0.1
    xx, yy = np.meshgrid(np.linspace(mins[0], maxs[0], resolution),
                          np.linspace(mins[1], maxs[1], resolution))
    Z = clusterer.predict(np.c_[xx.ravel(), yy.ravel()])
    Z = Z.reshape(xx.shape)

    plt.contourf(Z, extent=(mins[0], maxs[0], mins[1], maxs[1]),
                 cmap="Pastel2")
    plt.contour(Z, extent=(mins[0], maxs[0], mins[1], maxs[1]),
               linewidths=1, colors='k')
    plot_data(X)
    if show_centroids:
        plot_centroids(clusterer.cluster_centers_)

    if show_xlabel:
        plt.xlabel("$x_1$")
    else:
        plt.tick_params(labelbottom=False)
    if show_ylabel:
        plt.ylabel("$x_2$", rotation=0)
    else:
        plt.tick_params(labelleft=False)
```

Graficar datos nuevos

```
X_new = np.array([[ -0.5, 0], [0, 0.5], [1, -0.1], [2, 1]])
plt.figure(figsize=(6, 3))
plot_decision_boundaries(knn, X, show_centroids=False)
plt.scatter(X_new[:, 0], X_new[:, 1], c="b", marker="+", s=200, zorder
            =10)

plt.show()

labels = knn.predict(X_new)
print(labels)
```

¡Gracias!